Effective Fine-tuning for Low-resource Languages: A Case Study of Cangjie

Zhihao Lin†[1†], Zhaofeng Liu[1†], Mingyi Zhou[1*], Zihan Huang[1], Chi Chen[2], Wei Ma[3], Li Li[1*]

[1]Beihang University, Beijing, China.
[2]Fudan University, Shanghai, China.
[3]Singapore Management University, Singapore, Singapore.

*Corresponding author(s). E-mail(s): lilicoding@ieee.org;
Contributing authors: mathieulin@buaa.edu.cn;
†These authors contributed equally to this work.

Abstract

In recent years, large language models (LLMs) have emerged and reshaped software engineering tasks such as code completion. Although LLMs have shown great performance on code completion, their performance on low-resource languages like Cangjie shows significant limitations. The main challenge is the limited availability of training data for new programming languages, resulting in under-fitting on the syntax and semantic information of the code. In this paper, we introduce *CodeBridge*, an innovative approach designed for code completion tasks of low-resource programming languages like Cangjie. *CodeBridge* proposes a dual approach to leveraging cross-language knowledge: a three-stage continued pretraining strategy to explicitly transfer knowledge from similar high-resource languages, and a novel chain-of-thought data generation method that leverages LLMs' understanding of programming patterns combined with compiler feedback. Furthermore, *CodeBridge* proposes a prefix matching decoding strategy, which optimizes tokenization during inference to ensure consistency between training and inference. Our chain-of-thought approach addresses the fundamental trade-off between the high costs of generating compilable code and effective pretraining of code models. Our experiments demonstrate that *CodeBridge* achieves improvements in code completion for Cangjie at both line- and block-levels. Especially for DeepSeek-Coder-V2-Lite, our approach achieves 52.35% exact match at the line level and 33.27% line accuracy at the block level, representing improvements of 5.82% and 2.57%, respectively. In addition, the human-like chain-of-thought data provides an additional 1.01% improvement at

the block level. We open-source the entire training pipeline, including data collection, cleaning, three-stage training, and model inference, offering a validated framework to support code tasks on future new programming languages.

Keywords: Large Language Models (LLMs), code completion, Cangjie, low-resource programming languages, transfer learning, cross-language knowledge

1 Introduction

Large language models (LLMs), trained on vast corpora, have become increasingly important in software engineering (SE) [1]. Code-specific LLMs, built on extensive code datasets, have demonstrated remarkable capabilities in a range of SE tasks [2], such as code generation [3], summarization [4], and translation [5]. Developers leverage these models during the coding process [6], especially for repetitive and labor intensive tasks. As LLMs have shown high consistency with humans [7], they are widely used for code refactoring and code review [8]. Moreover, LLMs play a crucial role in debugging and ensuring the accuracy of the code [9]. Specifically, code completion is one of the most common tasks among these, as it provides recommendations for line- and block-level completions when developers are coding.

However, the performance of LLMs is highly affected by the quantity and quality of their training datasets. In high-resource programming languages (HRPLs) such as Python and Java, these models often achieve impressive precision in code completion, code generation, and related tasks [10–14]. In contrast, when it comes to low-resource programming languages (LRPLs), LLMs frequently struggle to attain similar levels of performance. This issue is particularly relevant for new and industrially significant languages. A key example is Cangjie [15], a modern programming language developed by Huawei for the OpenHarmony ecosystem [16]. The main challenge comes from two factors. First, the limited availability of training data for LRPLs prevents LLMs from fully learning the syntax and semantic information of the code. Second, in code completion scenarios, the last code token is often incomplete and may be tokenized incorrectly, affecting subsequent code generation. Although LLMs trained on high-resource programming languages develop tolerance to incorrect subtokens through extensive exposure, this tolerance is severely limited for LRPLs due to insufficient training data. Despite these limitations, many LRPLs are gaining popularity due to their advanced features, attracting substantial developers to use them. This is particularly true for Cangjie, whose deep integration with the OpenHarmony ecosystem highlights its potential for significant industrial impact. Regular compiler releases and sustained recent commits across multiple core repositories, iterated public documentation with an official organization presence for language docs, and actively maintained community tooling (tutorials, IDE integrations, setup actions) indicate the quick development of Cangjie community. In the OpenHarmony context, a strong execution base (active Ark components) and a growing set of third-party cases further underscore practical momentum. These quality- and pace-oriented indicators substantiate Cangjie's relevance and motivate our LRPL-oriented training–inference pipeline.

Therefore, developing effective AI-assisted tools for it is not just about supporting a low-resource language, but about providing critical AI infrastructure to empower a growing, strategic software ecosystem. This clearly emphasizes the urgent need to develop and train LLMs customized for these programming languages, even when datasets are limited [17].

Many researchers have focused on improving the performance of LRPLs [18–24]. In-context learning [25] is one way, which involves providing the LLM with additional information through crafted prompts, such as few shot [26] examples showcasing features of the targeted LRPL, to help the model better understand the language. Although context learning with real-time API documentation retrieval and compiler feedback can achieve high accuracy, it is impractical for the completion of production code due to prohibitive latency (3 minutes per sample) and cost ($0.5 per completion). Transfer learning [18, 24, 27] is another way, which enables LLMs to borrow knowledge from high-resource domains. However, existing approaches face three key challenges: (1) *How to maximize the benefits of transfer learning while minimizing negative transfer to the target LRPL?* Naive transfer may introduce irrelevant patterns that harm performance. (2) *How to generate high-quality training data for LRPLs when limited data prevents effective learning?* Simply increasing data quantity without ensuring quality and correctness is insufficient. (3) *How should we handle incomplete code tokens during inference?* In code completion scenarios, the last token is often partially typed and may be tokenized incorrectly, affecting generation quality. This is a problem that HRPLs can tolerate through extensive training, but LRPLs cannot.

To address these challenges, we adopt a *continued pretraining* paradigm that adapts existing large-scale code models rather than training from scratch. This design choice is motivated by fundamental practical and methodological considerations. First, for newly emerging languages like Cangjie with extremely limited data (approximately 8 million tokens), training models from scratch would require thousands of GPU hours and substantial financial investment while likely yielding inferior results due to severe data scarcity. Our continued pretraining approach, by contrast, leverages the extensive programming knowledge already encoded in existing foundation models (e.g., DeepSeek-Coder, CodeLlama, Qwen), requiring less GPU hours to achieve strong performance. Second, this paradigm shift reflects a different research question: rather than asking "how to build language-specific models given sufficient resources" (addressed by from-scratch methods such as CodeT5+ [28] and MultiCoder [20]), we focus on "how to efficiently adapt existing LLMs to new low-resource languages." This distinction is crucial for practical deployment scenarios where rapid support for emerging languages is needed without the massive infrastructure investment required for full-scale pretraining. Furthermore, from-scratch approaches like CodeT5+ (encoder-decoder architecture) and MultiCoder (Mixture-of-Experts) introduce architectural complexities: custom tokenizers, language-specific expert modules, and specialized training pipelines. These are difficult to justify when adapting proven decoder-only models can achieve strong performance with minimal overhead.

Within this continued pretraining paradigm, we propose *CodeBridge*, an innovative approach designed to improve code completion for low-resource programming

languages, taking Cangjie as a detailed case study. We propose a three-stage continued pretraining strategy to maximize syntax and semantic learning from limited training data. The strategy includes three phases: Exploration, Transfer, and Adaptation. (1) In the **Exploration Phase**, we use the Cangjie corpus with a relatively high learning rate to quickly learn the features of the language. (2) In the **Transfer Stage**, we perform transfer learning using high-quality Java and Rust corpora with a lower learning rate. These programming languages share similarities with Cangjie, enabling effective knowledge transfer. (3) In the **Adaptation Stage**, we use the same Cangjie corpus as in the first stage but with a smaller learning rate to fine-tune the integrated knowledge from all three languages. Concretely, we adopt learning rates of 2e-5 (exploration), 7e-6 (transfer), and 5e-6 (adaptation). We also propose a decoding strategy based on prefix matching to address the problem of the incorrect subtoken during inference. To further enhance the quality and correctness of generated code, we introduce a novel chain-of-thought data generation approach that leverages LLMs' understanding of similar programming languages combined with compiler feedback. This method generates high-quality training data by simulating the complete development process: from requirements analysis and API documentation retrieval, through initial code generation and compilation, to reflection and correction based on error feedback. We evaluated *CodeBridge* in line and block code completion tasks and found that *CodeBridge* achieved significant improvements. For example, for Deepseek-Coder-V2-Lite, *CodeBridge* has a 5.82% improvement for "Exact Match Rate" in line-level and 2.57% improvement for "Line Accuracy" in block-level over the method without transfer learning.

Our experimental design validates this continued pretraining strategy across multiple modern decoder-only architectures, demonstrating both its effectiveness and generalizability for the LRPL adaptation problem. Beyond standard continued pretraining, our design specifically targets LRPL challenges: (i) a stage-wise schedule tailored to scarce data (single vs. multi-chunk regimes, relative learning-rate magnitudes, and similarity-guided transfer language choice) to curb negative transfer and stabilize convergence; and (ii) a lightweight prefix-matching decoder that mitigates subtoken ambiguity for incomplete identifiers by aligning training and inference tokenization. This combination yields substantial line-level gains with minimal overhead and generalizes across different LLMs.

In summary, our work mainly has the following contributions:

- We propose a dual approach to leveraging cross-language knowledge for LRPLs: three-stage continued pretraining for explicit knowledge transfer and chain-of-thought data generation for implicit knowledge application with compiler feedback.
- We demonstrate that LLMs' extensive knowledge of high-resource programming languages can be effectively harnessed for low-resource languages through both training-time transfer and inference-time guidance.
- We validate the robustness and generalizability of *CodeBridge* by demonstrating consistent performance gains across multiple LLMs of varying architectures and sizes, offering an effective approach for code completion in low-resource settings.

- We open source our entire process in data crawling, data cleaning, model training, and inference. This may help future research on LRPLs. Our code and data are available in our repository: https://github.com/SMAT-Lab/CodeBridge.

2 Background

2.1 Code Completion

Code completion [29], often referred to giving completions based on previous code. This task is crucial to improving the code writing processes of developers by reducing errors and speeding up code writing [30]. It serves as a foundational task in code-related applications. Traditional approaches to code completion typically involve statistical language models and rule-based systems. The early approaches used n-gram models [29], probabilistic grammars, and pattern matching techniques to predict the next token or code fragment. These models, often trained on large code corpora [31], rely on the frequency of token sequences and are usually augmented by human experts to better capture the syntactic and semantic rules of programming languages. In contrast, LLM-based approaches have emerged as a powerful alternative in recent years [32]. Using advances in deep learning and transformer architectures, these models are trained on vast amounts of source code data to learn complex patterns and long-range dependencies. Unlike traditional methods, LLM-based models generate contextually relevant suggestions without the need for explicit rules of the programming language [33]. This allows them to adapt to different coding styles and environments, resulting in more flexible and accurate completions that can significantly enhance developer productivity.

2.2 Low-resource Programming Languages (LRPLs)

Programming languages are typically classified into high-resource programming languages (HRPLs) and low-resource programming languages (LRPLs). HRPLs, such as Python and Java, have extensive datasets available, which facilitate their representation in state-of-the-art (SOTA) LLMs. In contrast, LRPLs like Ruby, Swift, and Racket suffer from limited training data, leading to under-representation in LLMs and, consequently, reduced performance in software engineering tasks. Some LRPLs, including Cangjie, are not included in the pre-training phase of many LLMs, resulting in models that lack understanding of their syntax and semantics, which further diminishes performance. In addition, some domain-specific languages face the same limitations as LRPLs. The classification of HRPL, LRPL, and domain-specific languages is shown in Figure 1.

2.3 Cangjie

Cangjie [15] is a modern, multi-paradigm programming language developed by Huawei, designed to address the demands of intelligent applications across various scenarios. It integrates features from functional, imperative, and object-oriented programming, offering an efficient development experience. Cangjie emphasizes native intelligence, high performance, and robust security, making it particularly suitable for applications

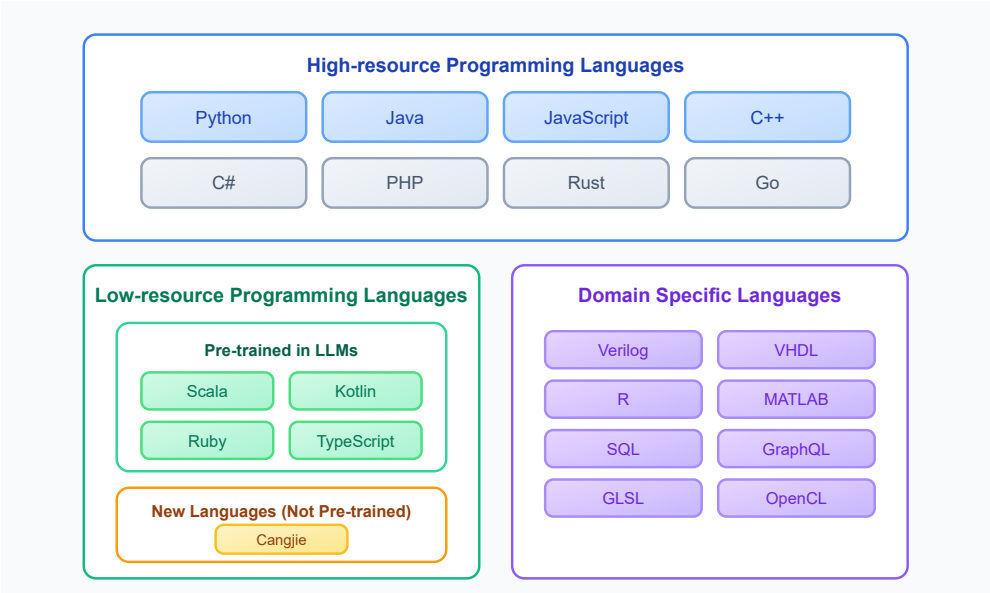

Figure 1 Classification of programming languages

within the OpenHarmony ecosystem [16]. This "native intelligence" is realized through features like its embedded AgentDSL (Agent Domain-Specific Language), which enables an organic fusion of natural and programming languages. The framework is designed to facilitate multi-agent collaboration and simplify symbolic expression, thereby supporting the development of diverse intelligent applications. For example, the code snippet shown in Figure 2 demonstrates how a 'Planner' agent can be defined and used in Cangjie.

```
// Definition of the Agent
@agent class Planner {
  @prompt[pattern=APE] (
    action: "Help the user plan a travel route",
    purpose: "To help the user visit more attractions with
        sufficient rest within the planned time",
    expectation: "Generate a reasonable travel itinerary,
        including timings, attractions, and commute information"
  )
}

// Usage of the Agent
let agent = Planner()
let result = agent.chat("I want to travel to Shanghai")
```

Figure 2 A Cangjie Code Example for AgentDSL.

The language supports advanced features such as type inference, pattern matching, and high-order functions, allowing developers to write concise and expressive code. Its concurrent object library simplifies the development of concurrent applications, enhancing resource utilization and performance. Additionally, Cangjie provides a comprehensive suite of tools, including debugging, static analysis, performance profiling, and testing frameworks, to support the entire software development lifecycle. In addition, it will be used in more and more OpenHarmony applications in the future.

However, as a new programming language, Cangjie currently lacks support across many domains, primarily due to the lack of datasets. This limitation is particularly evident in software engineering tasks that require high-quality datasets, especially those that leverage machine learning techniques, for example, using LLMs for code completion. However, with the development of the OpenHarmony ecosystem, Cangjie plays an increasingly significant role, attracting a growing number of developers. Therefore, the development of supporting tools and resources for Cangjie is becoming increasingly crucial.

2.4 Transfer Learning

Transfer learning [27] is a machine learning technique that adapts the model from one task to a different but related task. In code completion, this typically involves leveraging the knowledge from HRPLs to improve performance in LRPLs [34]. One of the fundamental challenges in applying deep learning models to LRPLs is the scarcity of high-quality training data [35]. Unlike HRPLs such as Python and Java, which benefit from vast repositories and high-quality open source code, LRPLs often lack sufficient publicly available datasets. This data imbalance leads to significant performance disparities in code completion task, as models pre-trained on HRPLs struggle to generalize effectively to LRPLs.

Several transfer learning techniques have been explored to bridge this gap. Fine-tuning a pre-trained model on a limited LRPL dataset is a common approach, but its effectiveness depends on the availability of high-quality examples. In addition, multilingual training strategies, in which models are trained on a mixture of HRPLs and LRPLs, have shown promise in improving LRPL performance. Recent advancements also explore semi-synthetic data generation, where models translate code from HRPLs to LRPLs and then validate the results through automated testing to provide more training data. These methods collectively demonstrate the power of transfer learning in enabling LLMs to perform well on LRPLs despite data limitations. Transfer learning mitigates the limitations of the LRPL code completion task by enabling knowledge reuse, allowing models to leverage patterns and structures learned from HRPLs and apply them to similar LRPLs with minimal additional training.

3 Motivation

Currently, low-resource programming languages (LRPLs) have a certain number of developers but lack corresponding effective code completion tools to assist developers. Thus, we focus on code completion for low-resource programming languages, which requires us to devote ourselves to improving effective training and inference of code

completion for low-resource programming languages. In the following, we will introduce what motivates our three-stage continued pretraining strategy and decoding strategy based on prefix matching.

The three-stage continued pretraining strategy is inspired by the language acquisition process. Figure 3 shows how Portuguese speakers learn Spanish. Since natural languages share significant structural and semantic similarities with programming languages, this process provides an intuitive yet effective framework for our three-stage continued pretraining strategy. When a Portuguese speaker initially learns Spanish, he first rapidly learns the basic grammar and vocabulary of Spanish. We call this step "learning". It is often said that it will be easier to learn a new language when you have already learned a language. The Portuguese speaker may then compare the new language (Spanish) he learns with a language he has learned (Portuguese) to help him explore the strengths and weaknesses, so that he will know what he needs to learn more in Spanish. For example, when he sees the sentence: "¿Cuánto tiempo hace que estudias español?", he will reflect on the similar representation in Portuguese: "Quanto tempo faz que estudas espanhol?". Although he may not fully remember this sentence in Spanish, he now knows the similarity and difference between these two languages. We call this step "transfer". Finally, he will precisely immerse himself in a Spanish-speaking environment to fine-tune his language skills, which means that he will further improve himself by strengthening strengths and overcoming weaknesses to adapt to Spanish. We call this step "adaptation". This motivates us to believe that it is the same as learning a new programming language. For example, when we learn a new programming language called Cangjie, we first directly learn its basic syntax and semantic knowledge. After the first learning, we may proactively compare with a programming language we have learned such as Java or Rust. Finally, we will learn Cangjie again. Thus, we propose a three-stage continued pretraining strategy.

The decoding strategy based on prefix matching is motivated by the phenomenon we have found during inference. As shown in Figure 4, when a code token (e.g., private) is incomplete in the input context, direct tokenization by the LLM might result in a subtoken (e.g., pri) that is out-of-context or misaligned with what the model learned during training. For high-resource programming languages, LLMs benefit from extensive training, developing robustness to such partial inputs. However, for LRPLs like Cangjie, where models are trained on limited data, their understanding of syntax and token patterns is less comprehensive. Consequently, an incorrectly interpreted prefix can significantly mislead the model, leading to irrelevant or erroneous completions. Our prefix matching decoding strategy (detailed in Section 4.4) is designed not only to handle prefixes but also to ensure that the model correctly processes these partial inputs by explicitly identifying the prefix and using it to guide the generation process. This provides crucial conditioning for the LLM, especially when its internal representations for the LRPL are not yet fully robust, thereby mitigating the negative impact of incorrect subtokens during inference.

STAGE 1: EXPLORATION (Reliance on Base Knowledge)

TARGET (Goal Spanish)

¿Cuánto tiempo hace que estudias español?

CURRENT (Base Portuguese)

Quanto tempo faz que...?
(Portuguese Logic applied blindly)

TARGET (Cangjie)

```
import std.sync.*
import std.time.*
main(): Int64 {
  spawn { => println("Thread") }
  return 0
}
```

CURRENT (Java Base)

```
new Thread(() -> {
  System.out.println("T");
}).start();

// Mental Model: Pure Java Objects.
// Unaware of 'spawn' keyword.
```

STAGE 2: TRANSFER (Hybrid Syntax / Interlanguage)

TARGET (Goal)

¿Cuánto tiempo hace que...?

CURRENT (Hybrid)

¿**Quanto** tiempo **faz** que...?
(Spanish Syntax + Port. Words)

TARGET (Idiomatic)

```
spawn { => sleep(100 * Duration.ms) }
```

CURRENT (Interlanguage)

```
main(): Int64 {
  spawn { =>
    System.out.println("T");
    Thread.sleep(100);
  }
  return 0
}
// Syntax shifted to Cangjie, but API
calls stuck in Java.
```

STAGE 3: ADAPTATION (Native Mastery)

✨ TARGET & CURRENT MERGED (Native Mastery)

✓ **Human Language:** ¿Cuánto tiempo hace que estudias español?

✓ **Code (Native Cangjie):**

```
import std.sync.*
import std.time.*
main(): Int64 {
  spawn { =>
    println("Running Native Thread")
    sleep(100 * Duration.millisecond)
  }
  println("Main Finished")
  return 0
}
```

Figure 3 The motivation of *CodeBridge*, paralleling human language acquisition with LLM fine-tuning. Layout: two stages on the first row (equal height) and one centered on the second row. Within each stage, Code Exploration is placed below Spanish Learning.

Figure 4 An Example of Prefix Matching

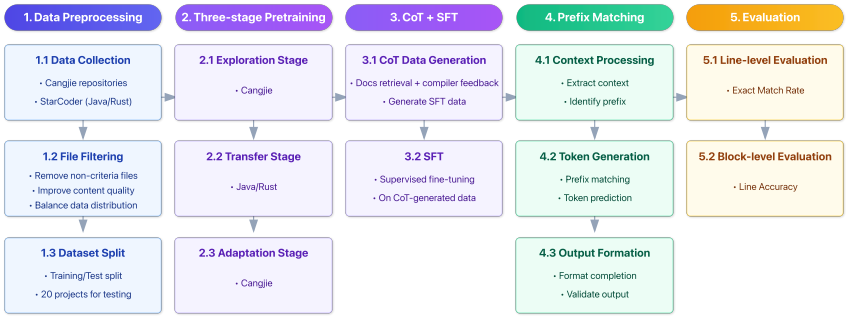

Figure 5 Overview of *CodeBridge*

4 Approach

4.1 Overview

This section provides a comprehensive overview of *CodeBridge*. As shown in Figure 5, *CodeBridge* consists of three key components. First, to acquire high-quality training data, *CodeBridge* employs data preprocessing rules to filter and clean the limited corpus available for low-resource programming languages. Second, the three-stage continued pretraining strategy is responsible for learning syntax and semantic information from the code of low-resource programming languages, including the exploration stage, transfer stage, and adaptation stage. Finally, the prefix matching-based decoding strategy ensures that the tokenization of the input during inference is consistent with that during training to correctly guide the inference generation, which includes context processing stage, prefix matching stage, and output formation stage.

4.2 Data Preparation

4.2.1 Data Pre-processing Rules

Due to the scarcity of available data for low-resource programming languages, acquiring high-quality pre-training data is of paramount importance. To this end, we employ a set of data pre-processing rules to get high-quality data.

First, we employ several rules for basic file filtering to remove files that do not meet fundamental criteria. The rules are as follows:

- **Type rule:** we filter files that belong to testing files to focus training on core, non-test logic.
- **Size rule:** we filter files that exceed a maximum size as extremely large files can be outliers, indicative of auto-generated content, or disproportionately affect training.
- **Encoding rule:** we just preserve files encoded in the form of 'UTF-8' to ensure consistent text processing.
- **Language rule:** we just preserve files that contain only Chinese and English characters to align with typical Cangjie source code character sets and avoid noise from other languages or encodings.
- **Max line/Max length/Average length rule:** If the lines of a file or the length of a file or the average length of a file exceed a maximum threshold, we filter the file. These rules help filter out atypical files, such as auto-generated code, data dumps, or files with excessive boilerplate, which might not be representative of typical coding patterns.

Second, we employ several rules to clean and improve the quality of the detailed content of the files. The rules are as follows:

- **Copyright rule:** we remove initial header comments containing the copyright notices in a file as these are typically not directly relevant to learning code structure.
- **Sensitization rule:** we eliminate sensitive information such as passwords and identification numbers for privacy and security considerations.
- **Format rule:** we standardize the code format by converting all newline characters to one standard, removing extra spaces at the end of each line, and removing any unnecessary blank lines. Standardization helps ensure consistency in the training data, reducing noise from stylistic variations.

Finally, we employ several rules to balance the data distribution. The rules are as follows:

- **Duplication rule:** we filter files that exceed a maximum similarity threshold to mitigate overfitting and ensure a more diverse training set.
- **Code ratio rule:** if the code ratio (code: comment) in a file exceeds a maximum ratio threshold, we preserve the file to prioritize files with a substantial amount of actual code relative to comments.

- **Hexadecimal/Digit character ratio rule:** if the hexadecimal character ratio or the digit character ratio in a file exceeds a maximum ratio threshold, we filter the file, as high proportions of such characters often indicate data files, obfuscated code, or auto-generated content rather than typical human-written source code, similar to filtering approaches used in large-scale code dataset curation.

4.2.2 Chain-of-Thought Data Generation with Compiler Feedback

Beyond preprocessing existing corpus, we also generate augmented training data to further enhance the model's ability to produce syntactically correct and functionally valid Cangjie code. We propose a data augmentation approach that leverages LLMs' understanding of similar programming languages (Java/Rust) combined with compiler feedback. While direct application of context learning with API retrieval and compiler feedback during inference can achieve high correctness, it is impractical for real-time code completion due to computational overhead and latency requirements. Our approach addresses this by using expensive context learning during training to generate high-quality supervision data, enabling the model to internalize the reasoning process. We use popular programming problems as seeds to ensure the generated training data covers commonly encountered coding patterns and challenges.

As illustrated in Figure 4.2.2, the pipeline is an iterative process that mirrors how developers transfer idioms between languages. The LLM first infers the required functionality from the completion context and consults Cangjie documentation to identify relevant APIs and syntax. It then emits an initial Cangjie adaptation of familiar programming patterns. The candidate code is compiled and compiler errors are collected. The model analyzes these error messages, re-checking documentation where necessary to correct mistaken assumptions. Finally, it produces a corrected, compilable solution together with the accompanying reasoning trace. By preserving both the mistakes and the corrections, each generated example teaches not only the final solution but also the pathway to arrive at it.

The key insight is that modern LLMs possess extensive knowledge of programming idioms from high-resource languages. This knowledge can be systematically transferred to Cangjie through three mechanisms. First, grounding in official documentation ensures the generated code conforms to Cangjie-specific APIs and syntax rather than simply imitating patterns from other languages. Second, compiler feedback provides concrete error signals that guide the refinement of cross-language knowledge and the correction of misconceptions. Third, explicit reasoning traces capture the entire problem-solving process, including mistakes and subsequent corrections, enabling the model to learn both correct solutions and the pathways to reach them.

This approach yields training data that are particularly valuable for low-resource programming languages with strict syntax requirements like Cangjie. The approach expands the effective corpus with diverse, realistic completion scenarios, instills error-handling and self-correction behaviors absent from clean code alone, and ensures that cross-language generalization is constrained by Cangjie's actual syntax and semantics through documentation retrieval and compiler validation.

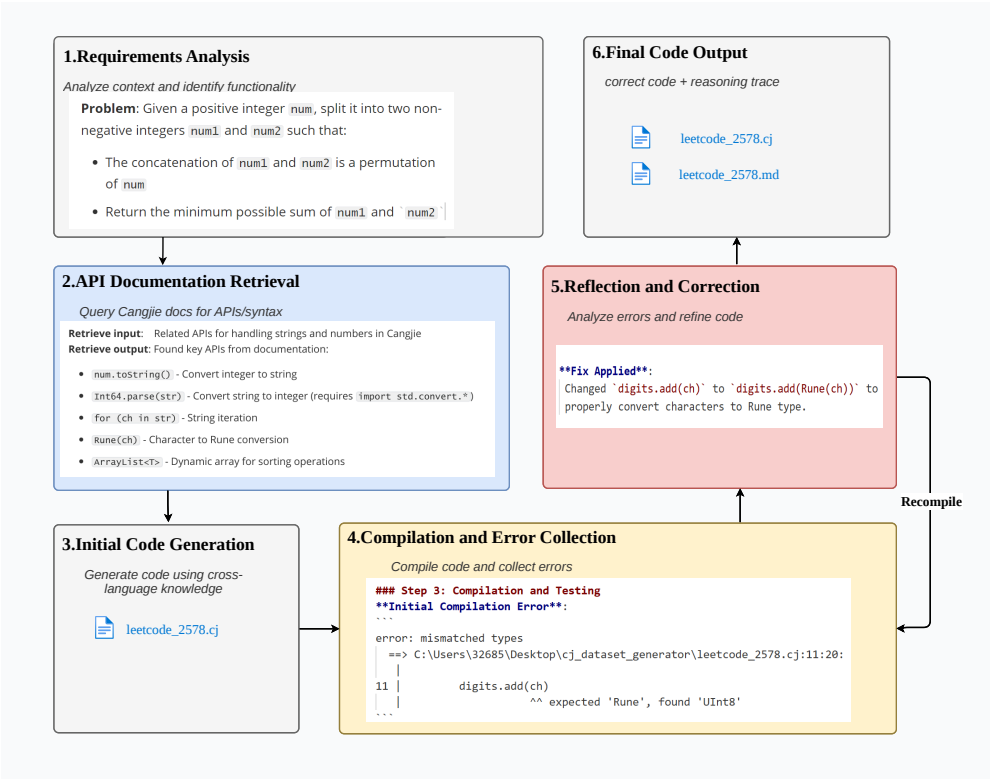

Figure 6 Chain-of-Thought Data Generation Process

4.3 Three-Stage continued Pretraining

As mentioned above, the three-stage continued pretraining is inspired by the process of language acquisition. The first stage is the exploration stage, which aims to directly learn the basic syntax and semantic knowledge of low-resource programming languages. The second stage is the transfer stage, which aims to transfer knowledge from other programming languages to low-resource programming languages. The third stage is the adaptation stage, which aims to further learn again to adapt the learned knowledge to low-resource programming languages.

Given an LLM pre-training as the base model, the exploration stage is first executed. The main purpose of the exploration stage is to let the LLM have an overview of the target low-resource programming language, such as Cangjie. The training data used in the exploration stage is provided by the data pre-processing stage to guarantee quality. We process files in a single-chunk mode, which means that each data sequence terminates at a file boundary without concatenation. Since the training data of the low-resource programming language are limited, there is no need to use multi-chunk mode (concatenate files) to reduce training time. In addition, though multi-chunk mode can help LLM to learn the relation across files, we assume that the ability of

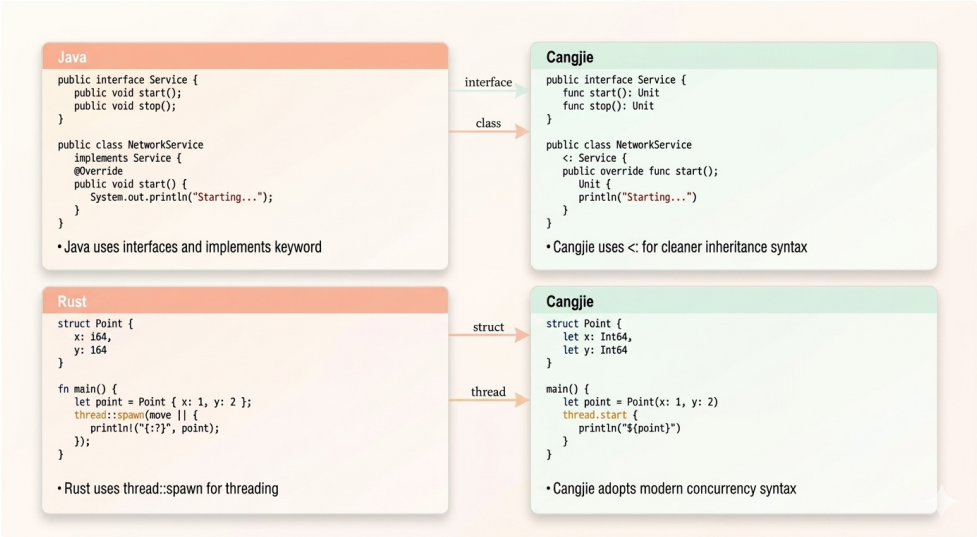

Figure 7 Comparison of Cangjie and Java/Rust

leveraging the relation across files can be transferred from other programming languages. We first want the LLM to learn the basic syntax and semantic information of the code in the low-resource programming language. We use a relatively high learning rate (2e-5 with 4 epochs) in this exploration stage, which is relative to the subsequent transfer and adaptation stages. The model can quickly and comprehensively acquire the basic knowledge of the target language in this stage.

After the exploration stage, the LLM is expected to have acquired a preliminary proficiency in code completion for the target low-resource language. The inclusion of a subsequent transfer stage is substantiated by prior studies [18, 24], which demonstrate the efficacy of cross-lingual transfer learning in low-resource scenarios. Based on semantic similarity analysis and data availability, we select Java and Rust as the transfer source languages (detailed language selection methodology is presented in Section 5.2.1).

In this state, we process files in a multi-chunk mode. We adopt an intermediate learning rate of 7e-6 for one epoch, which is lower than the exploration stage and slightly higher than the adaptation stage. It encourages knowledge transfer without overwhelming the target-language representations.

The third stage is the adaptation stage, which aims to integrate the knowledge learned from the exploration stage and the transfer stage and to learn the knowledge of the low-resource programming language in detail. As in the exploration stage, we also process files in a single-chunk mode in the adaptation stage. We use a smaller learning rate of 5e-6 with 4 epochs to carefully adapt and refine the integrated knowledge for the target language.

In summary, existing transfer learning approaches for LRPLs typically adopt one of two strategies: (1) *data-oriented* methods that augment LRPL corpora by translating code from HRPLs [18] or by selecting high-quality samples from large-scale mixed corpora [36], and (2) *architecture-oriented* methods such as Mixture-of-Experts that introduce language-specific modules [20]. Our three-stage strategy differs from both in a fundamental way: rather than modifying the training data or the model architecture, it restructures the *training schedule* itself to orchestrate how and when cross-language knowledge is acquired. Specifically, the Exploration phase first establishes a foundational representation of the target language, preventing catastrophic forgetting before transfer. The Transfer phase then introduces HRPL knowledge at a controlled learning rate, avoiding the instability that can arise from jointly mixing corpora with vastly different sizes. Finally, the Adaptation phase reconciles the transferred knowledge with the target language's specific patterns. This staged design, integrated with deliberate choices of learning rate (2e-5 / 7e-6 / 5e-6) and chunk mode (single / multi / single) per phase, directly addresses the risk of negative transfer that commonly plagues naive fine-tuning on mixed corpora. This effectiveness is evidenced by the performance drop observed when the transfer stage is placed before exploration (Table 4).

4.4 Decoding with Prefix Matching

During inference, we propose a decoding strategy based on prefix matching. As mentioned above, this decoding strategy is responsible for making sure that the tokenization of the input during inference is consistent with that during training to correctly guide inference generation. Although various constrained generation techniques exist for large language models, our method is specifically designed to address the problem of tokenization ambiguity for incomplete identifiers in low-resource languages. Unlike general constrained decoding methods that typically rely on a predefined vocabulary or grammar, our prefix matching technique dynamically filters candidate tokens at each generation step to ensure the generated sequence remains consistent with the user's partial input, thereby mitigating the risk of misdirection that can arise from the model's limited and unreliable knowledge of the language's vocabulary. The context processing stage in the decoding strategy aims at extracting previous code as input and prefix. The prefix matching stage in the decoding strategy aims to generate code completion of given input and prefix. The output format stage in the decoding strategy aims to format the code completion result.

In standard inference, the LLM generates the next token based on the complete previous code. However, this is not suitable for the low-resource programming language as mentioned before. Thus, separate the complete previous code into the previous code and prefix. The algorithm of the context processing stage is shown in Algorithm 1. Our decoding strategy extracts the prefix based on the trailing character of the input. If the input ends with a space, we backtrack from the end until we encounter the first space token, designating the segment from that point to the end as the prefix. Conversely, if the input ends with a non-symbol token, we backtrack from the last non-space character until we reach the first space or symbol token, and then continue until the first non-space token is encountered. This prefix matching strategy effectively

guides the LLM during inference, significantly improving the performance of code completion in the low-resource programming language.

Algorithm 1 Context Processing Stage

Require: inputText: a string
Ensure: (prev, prefix): two strings representing the previous text and the prefix

```
if inputText is empty then
    return ("", "")
end if
if the last character of inputText is a space then
    i ← length(inputText) − 1
    while i ≥ 0 and inputText[i] is a space do
        i ← i − 1
    end while
    prev ← substring of inputText from index 0 to i + 1
    prefix ← substring of inputText from index i + 1 to end
    return (prev, prefix)
else
    if the last character of inputText is not a symbol then
        i ← length(inputText) − 1
        while i ≥ 0 and inputText[i] is not a space and inputText[i] is not a symbol do
            i ← i − 1
        end while
        j ← i + 1
        while j < length(inputText) and inputText[j] is a space do
            j ← j + 1
        end while
        prev ← substring of inputText from index 0 to j
        prefix ← substring of inputText from index j to end
        return (prev, prefix)
    else
        return (inputText, "")
    end if
end if
```

Given the previous code and prefix, the prefix matching stage uses the LLM to generate tokens step by step. At each time step, the LLM generates a token list based on the previous code and already generated tokens. Then the token that combined with already generated and selected tokens that match the prefix will be selected. When the sequence of current generated tokens equals the prefix or starts with the prefix, we say that they are totally matched and that we do not need to consider the prefix at the following time steps. When the sequence of current generated tokens is just a substring of the prefix from the first character, we say that they are partially matched

and we need to continue to perform prefix matching until they are totally matched. If the prefix is empty at the beginning, we do not take the prefix into account.

The output format stage is responsible for converting the generated tokens into a sequence and removing the prefix. For example, as in the code shown in Figure 4, the previous code is

```
class Rectangle {
    var width: Number
    var height: Number
}
```

The prefix is 'pri', and we generate several candidates and select the candidate 'private' that started with the prefix. As motivated in Section 3, this decoding strategy is particularly vital for LRPLs. It ensures that the input provided to the LLM for generation is consistently structured, with the prefix correctly demarcated and utilized. This prevents the LLM, which may not have a robust understanding of the LRPL's token boundaries from limited data, from misinterpreting the initial part of an incomplete token and guides the generation process more effectively than direct, unassisted inference.

Figure 8 present an end-to-end flow code completion example: advanced context processing extracts the prefix and separates it from prior code; prefix-constrained filtering narrows candidates at each decoding step to those consistent with the user-typed prefix; finally, the model produces a function-level completion once the prefix is fully matched and the remaining decoding proceeds unconstrained.

It is worth noting that existing constrained decoding techniques for code generation, such as grammar-based decoding [37] and lexically constrained methods (e.g., grid beam search [38]), typically enforce output validity by restricting the decoding vocabulary according to a formal grammar or a predefined lexicon. While effective for well-specified languages, these methods require language-specific grammars or comprehensive vocabularies, which are often unavailable for LRPLs like Cangjie. Our prefix matching decoding is designed to address a fundamentally different problem: *tokenization misalignment* caused by incomplete identifiers at inference time. For HRPLs, LLMs develop sufficient robustness to tolerate such misalignment through extensive pre-training exposure. However, for LRPLs, even minor subtoken errors can severely mislead generation due to the model's limited familiarity with the language's token patterns. Our approach does not require any language-specific grammar or external lexicon; instead, it operates at the raw character level by extracting the partial prefix from the user's input and constraining only the first few decoding steps until the prefix is fully matched, after which unconstrained generation resumes. This makes it both lightweight and broadly applicable to any LRPL without additional linguistic resources.

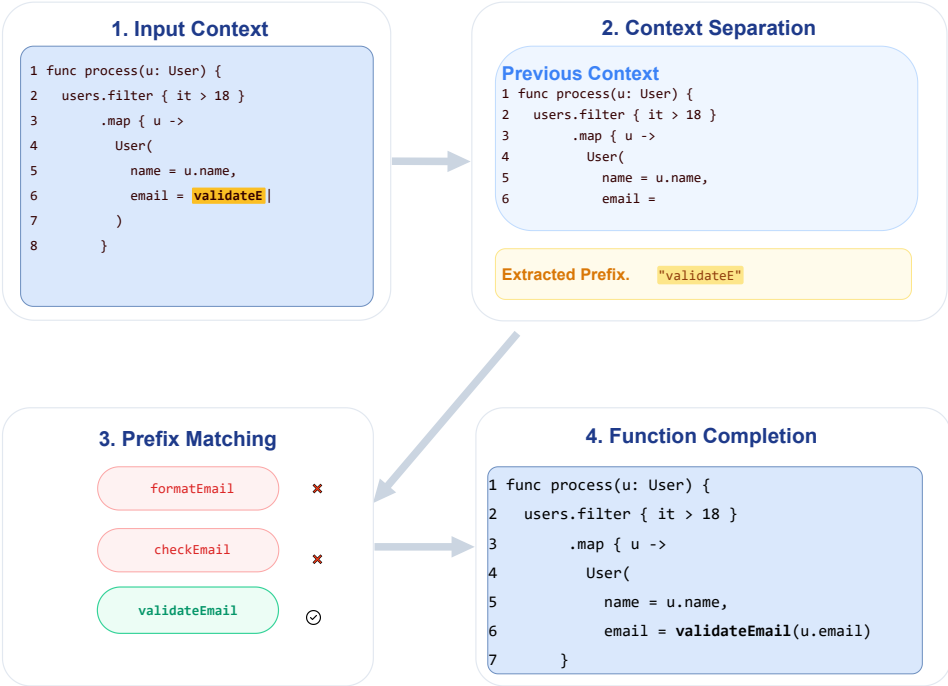

Figure 8 A code completion example illustrating advanced context processing, prefix-constrained candidate filtering, and the final function-level completion.

5 Evaluation

5.1 Research Question

To fully evaluate the performance of *CodeBridge*, we propose the following research questions (RQs):

RQ1: *How effective is* CodeBridge *for the completion of the Cangjie code?*

RQ2: *What is the impact of the transfer stage configurations on the performance of* CodeBridge *?*

RQ3: *Does* CodeBridge *remain effective when applied to various LLMs with different architectures and sizes?*

RQ4: *Can supervised fine-tuning (SFT) on chain-of-thought (CoT) data further enhance the performance of* CodeBridge*?*

In RQ1, we evaluate the performance of *CodeBridge* on the completion of the Cangjie code. We aim to determine the effectiveness of *CodeBridge*. Therefore, we conduct various experiments in RQ1 to show the benefits of *CodeBridge* using the three-stage continued pretraining strategy and the decoding strategy based on prefix matching. In RQ2, we explore the performance of the different configurations of the transfer stage, such as the training order of the transfer stage and the data volume

used in the transfer stage. In RQ3, we extend our evaluation to various architectures and LLM sizes to assess the generalizability of *CodeBridge*.In RQ4, we investigate whether augmenting *CodeBridge* with supervised fine-tuning on chain-of-thought data can further improve model performance. Specifically, we compare the baseline *CodeBridge* with an enhanced version trained with CoT-based SFT, aiming to examine if explicit reasoning traces and compiler-guided corrections provide additional benefits beyond the original pipeline.

5.2 Implementation

Our implementation was carried out on a 4×A100-80GB machine using DeepSpeed [39] for distributed training. The training process lasts for 320 GPU hours for Deepseek-V2-Coder-Lite. We begin with data collection. We primarily collect the Cangjie corpus from three repositories maintained by Huawei: Cangjie-SIG [40], Cangjie-TPC [41], and HW-PLLab [42].

5.2.1 Selection of Transfer Languages

The effectiveness of cross-language transfer critically depends on selecting source languages with high semantic similarity to the target language. To systematically identify the most suitable transfer sources for Cangjie, we developed a quantitative evaluation framework based on semantic similarity measurement.

Evaluation Methodology. We employed a *Semantic-Fixed evaluation method* to assess cross-language alignment. First, we systematically derived 41 core syntactic constructs from Cangjie's official language specification [15], covering fundamental programming paradigms including control flow (e.g., if-else, for, while), object-oriented features (e.g., class definitions, inheritance, interfaces), concurrency primitives (e.g., async/await, channels), type system features (e.g., generics, type inference), and error handling mechanisms. For each construct, we created minimal reference implementations in Cangjie and their corresponding implementations in six high-resource candidate languages: Java, Rust, Go, JavaScript, TypeScript, and C#. We then utilized GraphCodeBERT [43] to extract semantic embeddings of these implementations and performed cross-language code retrieval to quantify the "knowledge proximity" between Cangjie and each candidate language. Specifically, we measured similarity using three core retrieval metrics: **Recall@1** measures the proportion of queries where the correct match appears as the top-1 retrieval result; **Recall@5** measures the proportion where the correct match appears within the top-5 results; and **Mean Reciprocal Rank (MRR)** computes the average reciprocal of the rank at which the correct match is found. To provide a holistic similarity assessment, we aggregated these metrics into a **Composite Score** computed as:

$$\begin{aligned}\text{Composite Score} = {} & 0.35 \times \text{Recall@5} + 0.25 \times \text{MRR} \\ & + 0.25 \times \text{Alignment Margin} + 0.15 \times \text{Confusability}\end{aligned} \tag{1}$$

where Alignment Margin and Confusability are auxiliary metrics that measure embedding distance consistency and cross-language retrieval disambiguation, respectively.

Table 1 Quantitative Similarity Assessment between Cangjie and Candidate High-resource Languages. The Composite Score is weighted by Recall@5 (35%), MRR (25%), Alignment Margin (25%), and Confusability (15%).

Language Pair	Recall@1	Recall@5	MRR	Composite Score
Cangjie-Java	0.7805	0.9756	0.8653	**0.8331**
Cangjie-Go	0.7317	1.0000	0.8402	0.8299
Cangjie-Rust	0.8049	0.9512	**0.8720**	**0.8205**
Cangjie-JavaScript	0.7805	0.9268	0.8530	0.8128
Cangjie-TypeScript	0.7073	0.9512	0.8188	0.8117
Cangjie-C#	0.7805	0.9268	0.8466	0.8108

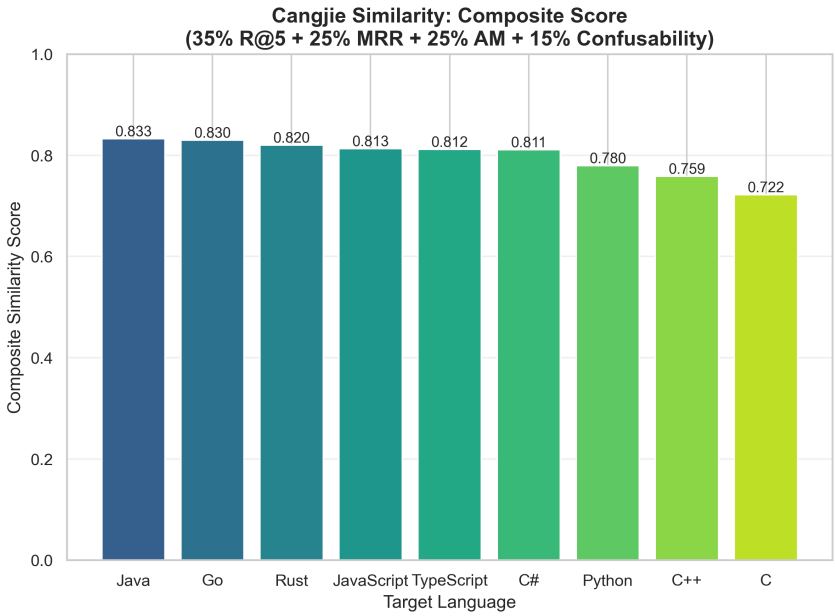

Figure 9 The Composite Similarity Score between Cangjie and Candidate High-resource Languages.

Language Selection Rationale. As shown in Table 1 and Figure 9, Java and Rust were identified as the optimal transfer sources based on both semantic correspondence and practical considerations. Java achieved the highest composite score (0.8331), indicating strong structural alignment with Cangjie's object-oriented paradigms and class-based logic. Rust demonstrated the highest Mean Reciprocal Rank (MRR) of 0.8720, reflecting significant conceptual parallels with Cangjie's modern systems-programming features and memory safety mechanisms. While Go exhibited a competitive composite score (0.8299), it was not selected as a primary transfer language for several reasons. First, as described in Section 2, Cangjie supports advanced features such as pattern matching and higher-order functions [15], which

are well-supported by Rust but absent or limited in Go. This makes Rust's abstractions more transferable to Cangjie than Go's simpler type system. Second, regarding the systems-level paradigm coverage that Go also targets, Rust provides a richer set of programming constructs (e.g., ownership system, enum-based error handling) that align more closely with Cangjie's modern design philosophy. Third, Go's error-handling model (explicit error returns) and concurrency model (goroutines and channels) differ substantially from Cangjie's approach, whereas Rust's async/await concurrency patterns share deeper conceptual analogies with Cangjie's concurrent object library. In summary, Java and Rust were prioritized because they jointly offer the broadest coverage of Cangjie's design features: Java covers the application-level, class-based, and interface-oriented paradigms, while Rust covers the systems-level, safety-oriented, and pattern-rich paradigms. Additionally, their status as high-resource languages provides substantially larger and more diverse training corpora, ensuring more robust and noise-resistant knowledge transfer signals.

Transfer Data Collection. For Java and Rust, we adopted the dataset provided by StarCoder [44], which has already undergone extensive deduplication and quality filtering during its training process. From this dataset, we selected data subsets corresponding to 8 million, 24 million, and 40 million tokens, respectively, based on repository popularity rankings (measured by GitHub stars). After data collection, we carefully perform a data pre-processing procedure using Python, applying file filtering, cleaning, and data balancing rules to ensure high-quality training data. After the data pre-processing procedure, we randomly split it into a training set and a test set. We randomly selected 20 projects for testing and used the rest for training to avoid overfitting. To further minimize data leakage, we ensured that these 20 test projects were distinct from training projects based on their source repositories and project names. While fine-grained, automated deduplication at the code-snippet level between training and testing sets is challenging for a new language like Cangjie due to limited specialized tooling, the project-level separation provides a reasonable safeguard against direct overlap. These 20 test projects were manually inspected and curated to cover a diverse range of typical Cangjie applications and language features observed in the collected corpus, with the aim of obtaining a representative evaluation benchmark. Then, an expert familiar with Cangjie syntax manually divided each code line in the test projects into three parts, the beginning, middle, and end. This manual process was guided by the principle of covering a wide range of Cangjie syntactic structures, including variable declarations, function calls, control flows, and its unique features, to ensure a comprehensive and representative test suite. This process produced 464 block-level completion samples and 447 line-level completion samples, allowing us to evaluate our method in both detailed and overall ways.

Then, we use a three-stage continued pretraining strategy (exploration, transfer, and adaptation) with tailored learning rates and training modes. We have tried multiple settings of different learning rates and epoch configurations to optimize the model's performance. Finally, we chose a learning rate of 2e-5 in the exploration stage, 7e-6 in the transfer stage, and 5e-6 in the adaptation stage with a 4+1+4 epoch schedule, which achieved the best results on our code completion task. Additionally,

we implemented a prefix matching decoding strategy to align inference tokenization with training, significantly improving code completion accuracy in low-resource programming languages.

We train with DeepSpeed (ZeRO-3), bf16, constant learning rate, AdamW optimizer (weight decay 0.01), per-device batch size 1 with gradient accumulation 16 (global batch 16), and cutoff length 4096. We use single-chunk packing in the exploration and adaptation stages, and multi-chunk packing in the transfer stage. Stage learning rates are selected via a small grid search around candidate values by monitoring training loss curves and held-out validation; we use a fixed 4+1+4 epoch schedule without early stopping. The final LRs are 2e-5 (exploration), 7e-6 (transfer), and 5e-6 (adaptation).

For the chain-of-thought data generation experiments in RQ4, we generated a dataset of 512 programming problems adapted from LeetCode easy-level questions. The problems were selected based on popularity ranking to ensure coverage of commonly encountered programming patterns. We employed Claude Sonnet 4.0 to systematically convert these problems to Cangjie syntax through our six-step CoT pipeline. Each generated example contains seven components: (1) problem description, (2) reasoning process, (3) API documentation retrieval, (4) initial code generation, (5) compiler error messages, (6) correction process, and (7) final compilable code. The generation process achieved a 100% compilation success rate, with all generated solutions passing the corresponding LeetCode test cases. Most problems required only one iteration of compiler feedback for successful correction, with API usage errors being the most common issue. The entire generation process required an average of 3 minutes and $0.5 per sample, totaling approximately 25.6 hours and $256 for the complete dataset.

To rigorously assess the performance of the Cangjie code completion model, we calculate the metrics that capture accuracy at both the line- and block-levels. Our choice of evaluation metrics is fundamentally constrained by the data scarcity inherent to low-resource programming languages. Unlike high-resource languages with extensive test suites and benchmarks, Cangjie's limited ecosystem means that comprehensive functional correctness evaluation through automated testing is not feasible at scale. The 464 block-level and 447 line-level samples in our test set, while carefully curated, represent the practical upper bound of what can be manually validated for a nascent language like Cangjie. More sophisticated evaluation approaches, such as execution-based testing or large-scale compilation verification, would require substantially more data and infrastructure than currently available for Cangjie. Therefore, this study focuses on Exact Match and Line Accuracy metrics, which serve as reliable proxies for code quality in data-constrained scenarios. These metrics directly measure the model's ability to generate syntactically correct and contextually appropriate code completions, which is the primary goal for practical code completion systems. While these metrics do not capture full semantic correctness, they provide meaningful insights into the model's understanding of Cangjie's syntax and programming patterns. For developers using code completion tools, receiving syntactically correct suggestions that match expected patterns significantly accelerates development, even if minor semantic adjustments are occasionally needed. The effectiveness of our approach within these

constraints demonstrates that meaningful progress can be achieved for LRPLs despite evaluation limitations imposed by data scarcity.

For **line-level tasks**, we use the following metric:

- **Exact Match Rate (EM):** This metric measures the percentage of instances where the generated code exactly matches the reference line. Provides a strict assessment of the model's ability to produce syntactically and semantically correct completions.

For **block-level tasks**, where the complexity and length of the generated code increase, we employ the following metric:

- **Line Accuracy (LA):** This metric evaluates the percentage of correctly generated lines within a block. It offers an overall measure of the quality of the reconstruction of the block.

5.3 The Effectiveness of *CodeBridge* (RQ1)

In this research question, we investigate the effectiveness of *CodeBridge*. We mainly focus on the impact of the three-stage continued pretraining strategy and the decoding strategy based on prefix matching. Furthermore, we evaluate different settings of the three-stage continued pretraining strategy to explore the effectiveness of our approach. We conducted several experiments to evaluate the different combinations of evaluation strategies. The different combinations of strategies are as follows:

1. **Base-Model:** Deepseek-Coder-V2-Lite-Instruct without any modification.
2. **Base+PM:** Base model with prefix matching decoding only.
3. **Explore+PM (Baseline):** Base model with "Exploration Stage" training and PM-decoding, serving as a normal pre-training baseline.
4. **Explore+Adapt+PM:** Adding "Adaptation Stage" to evaluate transfer impact.
5. **Explore+Transfer+PM:** Adding transfer stage to assess the impact of not having "Adaptation Stage".
6. **Full-Training:** Complete three-stage training without PM-decoding to evaluate the impact of PM-decoding in the Cangjie code completion scenario.
7. ***CodeBridge* (Full-Method):** Complete approach with all stages and PM-decoding.

Base-Model uses the original Deepseek-Coder V2-Lite-Instruct as a foundation. *Base+PM* applies only our prefix matching decoding strategy to the base model. *Explore+PM* adds exploration stage (continued pre-training on the Cangjie corpus) serving as our baseline. *Explore+Adapt+PM* and *Explore+Transfer+PM* evaluate individual stage contributions. *Full-Training* uses complete three-stage training to isolate PM decoding effects. *Full-Method* represents our proposed complete approach.

The evaluation results in Table 2 clearly demonstrate the effect of each component in our approach. First, the three-stage training strategy is core to enhancing the

Table 2 Effectiveness of Our Approach

	Line-Level (Exact Match Rate)	Block-Level (Line Accuracy)
Base-Model	5.50%	22.35%
Base+PM	35.49%	25.15%
Explore+PM (Baseline)	44.07%	31.94%
Explore+Adapt+PM	46.53%	30.70%
Explore+Transfer+PM	38.48%	28.58%
Full-Training	7.3%	29.28%
CodeBridge (Full-Method)	**52.35%**	**33.27%**

model's knowledge and capabilities, providing progressive improvements at both the line- and block-levels. The data in Table 2 show the distinct value of each phase on top of the PM-decoding baseline. The importance of the Exploration stage is evident when comparing "Base+PM" with our "Explore+PM (Baseline)". This single phase improves the line-level Exact Match (EM) by an absolute 8.58% (from 35.49% to 44.07%) and the block-level Line Accuracy (LA) by a significant 6.79% (from 25.15% to 31.94%). This confirms that building a foundational understanding of Cangjie's syntax and semantics is critical for both completion tasks. Furthermore, the Transfer and Adaptation stages build upon this foundation to achieve the best performance. Comparing the "Explore+PM (Baseline)" with the entire *CodeBridge*, these final two stages add another 8. 28% to the line-level EM and 1.33% to the block-level LA. This shows that transferring knowledge from related languages and then making adaptations to the target language is essential to achieve better performance, especially for fine-grained line-level completions.

Second, the PM-decoding strategy is equally crucial, as it solves the problem of how the model correctly "expresses" its knowledge. Its necessity is evident both before and after our three-stage training. For the untrained "Base Model", adding PM-decoding increases the line-level EM rate from 5.50% to 35.49%. More importantly, this necessity persists even after our three-stage pre-training. This is demonstrated powerfully by comparing the model after three-stage training without PM decoding ("Full training" achieving only 7. 3% EM) to our final *CodeBridge* with PM decoding (52. 35% EM). The absolute improvement of over 45% shows that while our training strategy is effective in teaching the model the syntax and semantics of Cangjie, it does not fully resolve the model's underlying sensitivity to inference-time tokenization errors. The PM-decoding strategy remains an indispensable component to ensure that this learned knowledge can be expressed correctly and robustly in the code completion task.

Therefore, our method is the result of two complementary components working together. The three-stage training is responsible for teaching the model the "knowledge" of the Cangjie language, while the PM-decoding strategy ensures the model can "express" this knowledge correctly in the code completion task. The results demonstrated that effective knowledge acquisition and robust knowledge expression are two essential and interconnected parts, both indispensable for unlocking high-performance code completion in the low-resource setting.

Answer for RQ1: Our complete three-stage strategy, incorporating "Exploration, Transfer and Adaptation" together with the decoding strategy based on prefix matching indeed make sense and achieve improvements in both the line-level and the block-level code completion tasks of Cangjie.

5.4 Transfer Stage Configurations (RQ2)

In this research question, we conduct a series of experiments to investigate how various configurations of the transfer stage affect the performance of *CodeBridge*.

Table 3 Impact of Transfer Data Volume on Model Performance

Transfer Data Volume	Line-Level (Exact Match)	Block-Level (Line Accuracy)
1:1 (8M tokens)	40.49%	32.00%
1:3 (24M tokens)	**52.35%**	**33.27%**
1:5 (40M tokens)	48.32%	32.10%

Table 4 Impact of Transfer Stage Sequence on Model Performance

Approach	Line-Level (Exact Match)	Block-Level (Line Accuracy)
CodeBridge (Full-Method)	**52.35%**	**33.27%**
Transfer+Explore+Adapt+PM	47.43%	31.91%

Firstly, the volume of data in the transfer stage plays a critical role. Insufficient data may hinder the full utilization of transfer learning information, while an excessively large dataset might cause the model to lean too heavily on the transfer corpus, thus overlooking the unique syntactic characteristics of Cangjie. Based on empirical observations, we compare three data scales, the first with an equivalent count as the Cangjie corpus (8,000,000 tokens), the second with three times the token count (24,000,000 tokens) and the last with five times the token count (40,000,000 tokens).

The results are shown in Table 3. From the results, we can see that a sufficient amount of transfer data can enhance the understanding of Cangjie by the model. For example, a dataset of 8 million tokens from Java and Rust yields poorer results compared to one with 24 million tokens, indicating that the model did not fully benefit from the transfer learning process. Conversely, a dataset with 40 million tokens also shows degraded performance, likely due to overfitting on Java and Rust. Secondly, we consider that the training order of the transfer stage is important. Thus we change the order from "Exploration, Transfer, Adaptation" to "Transfer, Exploration, Adaptation". The results are shown in Table 4. From Table 4, we can see that the transfer stage is suitable in the second stage instead of the first stage, which is consistent with our motivation mentioned earlier.

Answer for RQ2: The volume of data in the transfer stage and the training order of the transfer stage affect the performance. It is better to use a certain amount of transfer data avoiding both insufficiency and excess and to put the transfer stage in the second stage.

5.5 The generalizability of *CodeBridge* (RQ3)

From RQ1, we find that *CodeBridge* significantly improves the performance of the LLM in Cangjie, particularly for the completion of line-level code. To assess its generalizability, we compare four additional LLMs with different architectures and sizes. For architectural diversity, we selected Qwen-2.5-14b-Instruct [45] and CodeLlama-13b-Instruct [46], and for model size diversity, we chose Deepseek-Coder-1.3b and Deepseek-Coder-6.7b [47].

Table 5 Performance of *CodeBridge* on Different LLM Architectures and Model Sizes

Model	Training Step	Line-Level (Exact Match Rate)	Block-Level (Line Accuracy)
CodeLlama-13b-Instruct-hf	Base+PM	32.21%	27.24%
	Explore+PM	56.60%	33.68%
	Explore+Transfer+PM	50.56%	32.10%
	Explore+Adapt+PM	56.82%	33.90%
	CodeBridge (Full-Method)	**57.94%**	**34.13%**
Qwen2.5-14B-Instruct	Base+PM	29.69%	20.60%
	Explore+PM	46.98%	25.75%
	Explore+Transfer+PM	32.89%	21.89%
	Explore+Adapt+PM	48.10%	26.64%
	CodeBridge (Full-Method)	**50.56%**	**27.12%**
DeepSeek-Coder-1.3b-Instruct	Base+PM	27.46%	20.80%
	Explore+PM	43.53%	26.39%
	Explore+Transfer+PM	39.96%	22.83%
	Explore+Adapt+PM	43.97%	26.69%
	CodeBridge (Full-Method)	**44.20%**	**27.28%**
DeepSeek-Coder-6.7b-Instruct	Base+PM	32.37%	23.58%
	Explore+PM	51.79%	30.85%
	Explore+Transfer+PM (7e6 lr)	39.53%	24.60%
	Explore+Transfer+PM (1e6 lr)	44.42%	30.12%
	Explore+Adapt+PM	52.90%	30.94%
	CodeBridge (Full-Method, 7e6)	51.24%	28.65%
	CodeBridge (Full-Method, 1e6)	**54.02%**	**31.45%**
DeepSeek-Coder-V2-Lite-Instruct	Base+PM	35.49%	25.15%
	Explore+PM	44.07%	31.94%
	Explore+Transfer+PM	38.48%	28.58%
	Explore+Adapt+PM	46.53%	30.70%
	CodeBridge (Full-Method)	**52.35%**	**33.27%**

Based on the results shown in Table 5, we can derive the following insights about the generalizability of *CodeBridge*:

- **Overall Gains from the three-stage strategy continued pretraining strategy and the decoding strategy based on prefix matching.** By comparing each LLM's Base+PM with applying *CodeBridge* (Full-Method), we see consistent improvement in both line and block level, especially at the line level, which is consistent with what we find in RQ1. The improvements range from 16. 86% to 25. 73% on the line level and 6. 42% to 7. 87% on the block level.
- **Line-Level vs. Block-Level Performance.** While block-level metrics also improve, gains at the line level are more pronounced. This result indicates that

CodeBridge excels in fine-grained completions, which often require a precise fine-grained understanding of specific syntactic or semantic characteristics. Block-level code completion requires a global and general understanding, which requires a stronger LLM. Thus, we will leave this to our future work.

In addition, we note that DeepSeek-Coder-6.7b experiences significant performance drops when using the same learning rate with DeepSeek-Coder-V2-Lite-Instruct in the transfer stage. This indicates that the inappropriate learning rate in the transfer stage might have negative effectiveness. Carefully reducing the learning rate mitigates this problem.

In general, our experiments confirm that *CodeBridge* is effective in various large language model architectures and sizes. In addition, the appropriate hyperparameter tuning for different models can further enhance its performance.

> **Answer for RQ3:** *CodeBridge* can significant improve the performance of the code completion task for Cangjie in different architectures or sizes of LLMs, especially in the line level.

5.6 The effectiveness of chain-of-thought data (RQ4)

From RQ3, we have confirmed that *CodeBridge* is generalizable across different LLM architectures and sizes. In RQ4, we investigate the effectiveness of our chain-of-thought (CoT) data generation approach from two perspectives: (1) whether supervised fine-tuning (SFT) on CoT data can further improve model performance, and (2) the quality and characteristics of the generated CoT data. This dual analysis provides both quantitative performance evidence and qualitative insights into the CoT generation process.

5.6.1 Performance Improvement with CoT-based SFT

To evaluate the effectiveness of CoT data, we compare the baseline *CodeBridge* with *CodeBridge* + SFT on CoT-augmented data using CodeLlama-13b-Instruct and DeepSeek-Coder-V2-Lite-Instruct.

Table 6 Comparison of *CodeBridge* and *CodeBridge*+SFT

Model	Method	Line-Level (Exact Match Rate)	Block-Level (Line Accuracy)
CodeLlama-13b-Instruct	*CodeBridge*	57.94%	34.13%
	CodeBridge+SFT (CoT data)	**58.46%**	**35.14%**
DeepSeek-Coder-V2-Lite-Instruct	*CodeBridge*	52.35%	33.67%
	CodeBridge+SFT (CoT data)	**53.11%**	**34.13%**

Based on the results in Table 6, we can derive the following insights regarding the impact of SFT with CoT data: Compared to baseline *CodeBridge*, *CodeBridge* + SFT in CodeLlama-13b-Instruct achieves improvements of +0. 52% at the line level and +1. 01% at the block level. Although the gains are modest, they consistently demonstrate

that integrating CoT data into the fine-tuning stage can provide additional benefits. Interestingly, the relative improvement at the block level is greater than that at the line level. This indicates that CoT supervision helps the model reason over larger code spans and maintain contextual consistency across multiple lines.

To further understand the scalability of our CoT-augmented SFT, we analyze whether the current dataset size of 512 samples represents a performance saturation point. The consistent improvements observed across both line-level and block-level metrics, without a discernible plateau in performance gains, suggest that the model's reasoning capacity for Cangjie patterns has not yet been fully exhausted. Currently, the CoT dataset is primarily constructed from fundamental algorithmic patterns. We anticipate that expanding this corpus to encompass more complex, Cangjie-specific idioms, such as those involving its unique concurrency model, specialized library APIs, and advanced memory management constructs, would likely yield more substantial performance breakthroughs. While the current scale of 512 samples reflects a pragmatic balance between data diversity and the non-trivial computational cost of high-fidelity CoT generation (averaging 3 minutes and $0.5 per sample), our findings provide a strong empirical motivation for scaling this approach. Future research could investigate more resource-efficient strategies, such as distilling reasoning traces from teacher models or incorporating iterative compiler feedback, to further expand the CoT dataset without prohibitive overhead.

5.6.2 Analysis of CoT Data Generation Quality

To better understand the failure patterns and limitations of our CoT data generation process, we conducted a systematic analysis of the 512 generated Cangjie code samples. Among these, **314 samples (61.3%)** were successfully compiled in the first attempt without requiring any correction, while **198 samples (38.7%)** underwent at least one round of compiler-guided correction. The iterative correction process demonstrated rapid convergence, with an average of only **0.50 correction rounds** per sample. The distribution of correction rounds was as follows: 161 samples (31.4%) required exactly one round, 20 samples (3.9%) required two rounds, and only 17 samples (3.3%) required three or more rounds. This high first-attempt success rate and low average iteration count indicate that the LLM possesses substantial inherent capability to generate syntactically valid Cangjie code, while compiler feedback effectively resolves the remaining discrepancies.

We further categorized the error types that necessitated correction, as summarized in Table 7. **Syntax errors (30.9%)** constituted the most frequent category, often arising from subtle differences between Cangjie and source languages (Java/Rust) in structural conventions such as bracket usage or keyword placement. **API misuse (22.8%)** typically occurred when the model incorrectly mapped familiar library functions from high-resource languages to Cangjie's standard library. **Type mismatches (21.7%)** reflected challenges in adapting to Cangjie's strict type system, particularly in scenarios involving generic types or implicit casting. **Logic errors (10.7%)**, though less frequent, were more complex to resolve as they involved algorithmic flaws rather than syntactic issues. The remaining **other errors (14.0%)** primarily consisted of compiler warnings related to code quality issues, such as unused variables.

Table 7 Distribution of Error Types in CoT Data Generation

Error Type	Count	Ratio (%)
Syntax error	84	30.9
API misuse	62	22.8
Type mismatch	59	21.7
Logic error	29	10.7
Other	38	14.0

```
func splitNum(num: Int64): Int64 {
    let numStr = num.toString()
    let digits = ArrayList<Rune>()

    // Extract digits with type-safe conversion
    for (ch in numStr) {
        // [Initial Error]: digits.add(ch)
        // [Compiler]: expected 'Rune', found 'UInt8'
        digits.add(Rune(ch)) // [Correction]: Explicit conversion
    }

    let sortedDigits = sortDigits(digits)
    var num1Str = ""; var num2Str = ""

    for (i in 0..sortedDigits.size) {
        if (i % 2 == 0) {
            num1Str += sortedDigits[i].toString()
        } else {
            num2Str += sortedDigits[i].toString()
        }
    }

    let num1 = if (num1Str.size > 0) { Int64.parse(num1Str) }
        else { 0 }
    let num2 = if (num2Str.size > 0) { Int64.parse(num2Str) }
        else { 0 }
    return num1 + num2
}
```

Figure 10 Example of compiler-guided correction in the CoT dataset (Split With Minimum Sum).

To further illustrate the typical correction patterns, we present a concrete example from our CoT dataset in Figure 10. The task is to solve LeetCode 2578 (Split With Minimum Sum), which requires splitting a positive integer's digits into two numbers with minimal sum. The LLM correctly identified the algorithm (sort digits, distribute alternately) and retrieved relevant Cangjie APIs (`num.toString()`, `Int64.parse()`, `ArrayList<Rune>`). However, the initial implementation contained a type error: the

code attempted to add string-iteration characters directly to an `ArrayList<Rune>`, as Cangjie's `for (ch in str)` yields `UInt8` rather than `Rune`. The compiler reported: `expected 'Rune', found 'UInt8'`. The model resolved this with a single-line fix by changing `digits.add(ch)` to `digits.add(Rune(ch))`, after which the corrected code passed all test cases.This example is representative of the most common error category (syntax/type errors, 30.9%), where the LLM's cross-language intuition produces nearly correct code that requires only minor type conversion adjustments specific to Cangjie's type system.

The error distribution reveals that the majority of corrections addressed **language-specific syntactic and API usage issues** rather than fundamental conceptual misunderstandings. This pattern suggests that large language models already possess strong cross-language programming intuition, but require explicit compiler feedback to fine-tune their knowledge to Cangjie's specific syntax and semantics. Our findings validate the design choice of integrating compiler-based self-correction, which effectively bridges cross-language discrepancies with minimal iteration overhead.

> **Answer for RQ4:** Our chain-of-thought data generation approach proves effective from both quantitative and qualitative perspectives. Supervised fine-tuning on CoT data provides additional performance gains (+0.52% line-level, +1.01% block-level), particularly improving block-level completion by helping models reason over larger code spans. Quality analysis reveals that the CoT generation process achieves 61.3% first-attempt compilation success with rapid convergence (0.50 rounds on average), and compiler feedback effectively addresses language-specific syntactic and API usage issues (constituting 75.4% of errors), validating that LLMs possess strong cross-language programming intuition that can be refined through compiler-guided self-correction for low-resource languages like Cangjie.

6 Related Work

6.1 Code Completion and Code Generation

Recent advances in machine learning have markedly improved code generation and completion. Large-scale transformer-based models, when fully trained on extensive code datasets, exhibit an impressive ability to generate code [48]. For example, OpenAI Codex fine-tuned GPT-3 on billions of GitHub code lines, dramatically improving performance on coding benchmarks and increasing the popularity of GitHub Copilot among developers [31]. In addition to general-purpose LLMs, researchers have developed specialized models tailored for coding tasks. CodeT5, a pre-trained encoder–decoder transformer augmented with code-specific objectives, exemplifies this focus [49]. Similarly, AlphaCode addresses competitive programming challenges by generating and filtering large sets of candidate solutions, while CodeLlama has also demonstrated robust performance in various code-related tasks [46].

In addition, reasoning models such as Deepseek-R1 [50] and GPT-o1 have demonstrated even stronger capabilities in code generation and completion. Using these

powerful models, tools like Manus can perform code generation tasks with remarkable efficiency, achieving performance that is nearly on par with that of human developers. Recent developments also highlight the integration of code synthesis capabilities into software development environments, providing context-aware, real-time suggestions. This integration not only accelerates the coding process, but also facilitates learning by offering developers immediate examples and explanations of best practices and common programming patterns. For instance, Cursor has reshaped developers' workflows by reducing time spent on repetitive coding tasks.

Another promising avenue in code generation is multimodal models, which combine code with additional context such as natural language instructions, diagrams, or execution traces to enhance accuracy and clarity. For example, state-of-the-art LLMs like GPT series and Claude series have shown a strong ability to generate code that aligns closely with developers' intents.

6.2 LLM for LRPLs

LLMs for LRPLs face significant challenges due to the scarcity of training data. To address this, one promising approach is the generation of synthetic data in conjunction with program repair. For example, Mora et al. [35] introduce an intermediate language framework that takes advantage of HRPLs such as Python to guide LLMs in generating syntactically correct code for LRPLs. By employing compiler-based repair techniques to fix syntactic errors in generated output, this method not only expands the training dataset but also improves the overall success rate of code generation.

Another strategy involves efficient continued pretraining (CPT) tailored for low-resource languages. Xie et al. [36] focus on selecting high-quality language-specific data from large corpora using a combined scoring method. They use global scoring to evaluate the general importance of a sentence and use local scoring to assess its immediate context. This dual approach ensures the inclusion of contextually rich and representative sentences. Nag et al. [51] extend this idea by augmenting the model vocabulary with new tokens for underrepresented words, significantly enhancing the model's ability to understand and generate rare words, even with limited training data.

A further promising direction is multi-programming lingual pre-training combined with Mixture-of-Experts (MoE) architectures. The MultiCoder model [20] exemplifies this approach by integrating language-specific experts with those of other languages, thus capturing language-specific and cross-language patterns. This mechanism has notably improved code completion performance for languages like Ruby and Racket, where available training data is scarce.

Additionally, enhancing code generation through advanced fine-tuning and in-context learning methods has shown considerable promise. Joel et al. [22] compare classic fine-tuning with several in-context learning variants. Although fine-tuning adapts model weights directly to the target language, it is computationally expensive and heavily dependent on data quality. In contrast, in-context learning leverages carefully crafted prompts to provide supplementary examples, offering a cost-effective alternative that becomes increasingly advantageous for larger models.

Finally, the transfer of learning from high-resource to low-resource languages offers a viable solution to bridge performance gaps. Cassano et al. [18] propose a method

that generates semi-synthetic training data by translating high-quality code from an HRPL (e.g., Python) into a target LRPL (such as Julia, Lua, OCaml, R, or Racket). By validating the translated code through automated test cases and compilers, this approach ensures the production of high-quality synthetic data for further training.

7 Discussion

Our work demonstrates two complementary approaches to leveraging cross-language knowledge for low-resource programming languages: the three-stage continued pre-training strategy and the chain-of-thought data generation method. These approaches address different aspects of the challenge and can be used synergistically.

7.1 Efficiency and Practicality of Prefix Matching

The prefix matching decoding strategy is designed to address tokenization ambiguity for incomplete identifiers in low-resource languages while maintaining practical deployment feasibility. The context processing algorithm (Algorithm 1) operates in $O(n)$ time complexity, where n is the length of the input text, requiring only a single backward scan to identify prefix boundaries. During generation, prefix matching adds a constant-time string comparison at each decoding step, which is negligible compared to the forward pass of the LLM. In practical deployment within IDEs, this overhead is imperceptible to users, typically adding less than 1ms to the overall completion latency. The primary computational cost remains dominated by the LLM inference itself, making our prefix matching strategy suitable for real-time code completion scenarios without compromising user experience.

7.2 Evaluation Constraints and Future Directions

The evaluation methodology in this work reflects the fundamental challenges of conducting rigorous research on low-resource programming languages. Our reliance on textual similarity metrics is not a methodological choice but a practical necessity imposed by data constraints. The limited availability of Cangjie code, combined with the absence of comprehensive test suites and benchmarks, makes large-scale functional correctness evaluation infeasible. This constraint is not unique to our work but represents a systemic challenge in LRPL research that the community must address collectively. Our chain-of-thought approach provides a pathway toward more comprehensive evaluation by generating compiler-validated examples and complete reasoning traces. By training on such high-quality data, the model learns to generate code that is more likely to compile successfully and avoid common syntax errors. The practical value of our approach is evident even within current evaluation constraints: syntactically correct and contextually relevant suggestions significantly reduce developer workload, even when minor semantic adjustments are occasionally needed. Future work should focus on developing automated testing frameworks and expanding code corpora for Cangjie, which would enable more comprehensive functional correctness evaluation and unlock the full potential of AI-assisted development for this strategic programming language. A primary limitation of our current evaluation is its reliance on textual

similarity metrics, such as exact match and line accuracy, which do not directly measure the functional correctness of the generated code. However, our chain-of-thought data generation approach provides a clear pathway toward this goal by emphasizing syntactic and semantic validity during training. By training the model on compiler-validated examples derived from an iterative correction process, it implicitly learns to avoid common syntax and API usage errors, thereby increasing the likelihood of producing compilable code. Consequently, even when the generated code requires minor functional corrections, its practical value is substantial; receiving syntactically correct and contextually relevant suggestions significantly reduces developer workload. Future work should therefore focus on developing automated testing frameworks for Cangjie to enable direct functional correctness evaluation, building on the strong syntactic foundation established by our approach.

7.3 Cross-Language Knowledge Transfer

Our work highlights the nuanced nature of programming knowledge transfer, which *CodeBridge* harnesses through both explicit and implicit mechanisms. The three-stage pre-training strategy facilitates **explicit transfer** by directly adapting the model to Cangjie through continued training on corpora from similar, high-resource languages. In contrast, the chain-of-thought generation method relies on **implicit transfer**, leveraging the large language model's inherent understanding of universal programming patterns and guiding it to apply this latent knowledge effectively through documentation retrieval and compiler feedback. Crucially, both approaches are refined by **error-driven learning**; the former uses training loss gradients as an error signal, while the latter employs explicit compiler feedback to iteratively correct and improve code generation. The success of these complementary strategies validates our core hypothesis: modern LLMs possess extensive cross-language programming knowledge that can be effectively unlocked and applied to benefit low-resource programming languages.

A natural question is how well *CodeBridge* would generalize to LRPLs that are less similar to Java and Rust. We note that the framework consists of two components with different generalization properties. The prefix matching decoding strategy is inherently language-agnostic: it operates at the character level without requiring any language-specific resources, and is therefore equally applicable to any LRPL regardless of its similarity to high-resource languages. The three-stage pretraining strategy, however, relies on cross-language transfer whose effectiveness depends on the availability of structurally similar HRPLs. For LRPLs that share paradigms with well-represented languages (e.g., a functional language similar to Haskell, or a scripting language similar to Python), we expect similar gains. For LRPLs with fewer high-resource counterparts, the transfer stage may yield diminishing returns; in such cases, the exploration and adaptation stages, along with the prefix matching decoding, would still provide meaningful improvements over direct fine-tuning, as demonstrated by the results in Table 2 (Explore+Adapt+PM" vs. Base-Model"). Furthermore, the chain-of-thought data generation pipeline, which relies on documentation retrieval and compiler feedback rather than cross-language similarity, can serve as an alternative knowledge

source when suitable transfer languages are unavailable. We leave systematic validation across diverse LRPLs to future work.

7.4 Future Directions

Building upon our findings, an immediate direction for future research is the development of automated testing frameworks for Cangjie. Such frameworks would not only enable the direct evaluation of functional correctness—a limitation of current textual similarity metrics—but also unlock novel training paradigms through automated test-case generation. To further enhance the quality of training data, one could explore multi-agent systems where specialized agents collaborate on API retrieval, code generation, and error correction, evolving the chain-of-thought process we introduced. Beyond Cangjie, a key avenue for investigation is the generalization of the *CodeBridge* methodology to other low-resource programming languages, as its core principles are designed to be broadly applicable. Finally, deploying these models in real-world Integrated Development Environments (IDEs) to study their impact on developer productivity would provide invaluable insights for practical application and adoption.

8 Threats to the validity

8.1 Threats to the internal validity

First, the training data of Cangjie is indeed very little, which may cause the problem of overfitting. However, we did not set too many epochs for training the Cangjie corpus to avoid overfitting. In addition, the test set we use for the evaluation is between projects, which can avoid testing cases that may be overfitting. Second, the programming languages used for the transfer stage are decided by expert experience. However, the results of the evaluation show that it works. In addition, if there exist other programming languages that can be used for the transfer stage, we think it is easy to add. Third, the hyper-parameters (such as learning rate) used for training may not be the best. These parameters were determined through iterative experimentation in a dedicated validation set. We acknowledge that a more systematic and exhaustive hyperparameter optimization protocol could potentially yield further performance improvements. However, the empirical results confirm that our selected configuration is effective and provides a strong and validated baseline for the task. Fourth, our evaluation relies on textual similarity metrics rather than functional correctness measures. This limitation stems directly from the data scarcity that defines low-resource programming languages: the limited Cangjie corpus and absence of comprehensive test suites make large-scale functional evaluation infeasible. While more sophisticated evaluation approaches would be preferable, they require substantially more data and infrastructure than currently available. Our chosen metrics provide meaningful insights within these constraints and represent the current state-of-the-art for LRPL evaluation. While we acknowledge that reporting statistical measures would strengthen the reliability of our findings, the consistent performance trends across different model architectures (RQ3) and the substantial margin of improvement over baselines provide

evidence of the robustness of our approach. Future work could incorporate multiple training runs with different random seeds to quantify variance. Last but not least, the semantic similarity evaluation in Section 5.2.1 relies on GraphCodeBERT [43] to measure cross-language similarity between Cangjie and candidate transfer languages. GraphCodeBERT was pre-trained on the CodeSearchNet corpus, which covers only six programming languages: Python, Java, JavaScript, PHP, Ruby, and Go. Since Rust, C#, and Cangjie are not included in its pre-training data, the semantic embeddings and similarity scores for these out-of-distribution languages may be less reliable than those for the in-distribution languages. Despite this limitation, we believe the evaluation still provides meaningful guidance for several reasons. First, the relative ranking is largely consistent with linguistic intuition: Java, which is in-distribution, achieves the highest Composite Score, while Rust still obtains a competitive score due to its structural similarities with in-distribution languages such as Java and Go, although it is technically out-of-distribution. Second, the subsequent transfer learning results (RQ1 and RQ3) empirically validate the effectiveness of the selected transfer languages, suggesting that the similarity-based selection, even if imperfect, yields practical benefits. Nevertheless, we acknowledge that a model pre-trained on a broader set of languages could provide more accurate similarity estimates, and we encourage future work to explore such alternatives.

8.2 Threats to the external validity

Our work focuses primarily on the low-resource programming language Cangjie, serving as an in-depth case study. Although we hypothesize that the underlying principles of *CodeBridge* (staged learning and guided decoding for under-resourced models) could be beneficial for other LRPLs, particularly those exhibiting similar characteristics such as recent emergence or limited corpus size, further extensive studies are required to confirm its generalizability across different languages and application areas. More studies are needed to confirm whether our approach can be applied effectively to other languages and different application areas. In addition, since there are too many LLMs, it is very difficult for us to validate our approach for each LLM. However, the results of the evaluation in RQ3 prove that our approach can be applied to different LLMs to some extent. Furthermore, as discussed in the Introduction, our evaluation focuses on continued pretraining methods and does not include comparisons with from-scratch training approaches like CodeT5+ [28] and MultiCoder [20], as these methods address fundamentally different research questions and face prohibitive computational costs for newly emerging languages with limited data.

9 Conclusion

In this paper, we present *CodeBridge*, a comprehensive framework for effective code completion in low-resource programming languages, with Cangjie as a detailed case study. Our work advances AI-assisted development for emerging languages by combining explicit cross-language transfer with compiler-guided data generation. Specifically, we explore a dual strategy to utilize LLM's prior knowledge: a three-stage continued pretraining process that transfers patterns from high-resource languages such as

Java and Rust, and a chain-of-thought data generation pipeline that integrates documentation retrieval and compiler feedback to create reasoning-rich training examples. This combination addresses the inherent trade-off between efficiency and correctness. While context learning approaches are often accurate but computationally expensive, and pretraining approaches are efficient yet prone to errors, our method harnesses expensive compilation feedback during training so that inference remains lightweight while code quality improves substantially. Beyond syntactic similarity metrics, our approach also lays the foundation for more meaningful evaluation by teaching models not only to produce correct code but also to reason through errors, analyze compiler diagnostics, and iteratively refine solutions. Experimental results show consistent improvements in line- and block-level completion across different model architectures and sizes, underscoring the practical value of our approach. More broadly, our findings suggest that the core challenge for low-resource programming languages lies not in the absence of programming knowledge in modern LLMs but in the mechanisms for transferring and grounding that knowledge. The techniques we propose for Cangjie are therefore extensible to other emerging languages, especially those with structural or semantic parallels to well-established ones. By bridging the gap in AI-powered tooling for LRPLs, this work contributes not only to a concrete methodology and open source implementation, but also to a vision for empowering developer communities and accelerating the adoption of strategic programming languages. Future directions include advancing functional correctness evaluation, integrating automated testing frameworks, and deploying these methods in real-world development environments.

Declarations

Funding

Not applicable.

Ethical Approval

Not applicable.

Informed Consent

Not applicable.

Author Contributions

Zhihao Lin: Conceptualization, Methodology, Software, Investigation, Writing – Original Draft. Zhaofeng Liu: Conceptualization, Methodology, Software, Investigation, Writing – Original Draft. Mingyi Zhou: Investigation, Data Curation. Zihan Huang: Investigation, Data Curation. Chi Chen: Writing – Review & Editing, Supervision. Wei Ma: Writing – Review & Editing, Supervision. Li Li: Conceptualization, Methodology, Resources, Writing – Review & Editing, Supervision, Project Administration, Funding Acquisition. All authors reviewed the manuscript.

Data Availability Statement

The training pipeline, evaluation benchmark, and experimental data supporting this study are available in our repository: https://github.com/SMAT-Lab/CodeBridge.

Conflict of Interest

Not applicable.

Clinical Trial Number

Clinical trial number: not applicable.

References

[1] Fan, A., Gokkaya, B., Harman, M., Lyubarskiy, M., Sengupta, S., Yoo, S., Zhang, J.M.: Large language models for software engineering: Survey and open problems. 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE), 31–53 (2023)

[2] Zheng, Z., Ning, K., Wang, Y., Zhang, J., Zheng, D., Ye, M., Chen, J.: A survey of large language models for code: Evolution, benchmarking, and future trends. arXiv preprint arXiv:2311.10372 (2023)

[3] Du, X., Liu, M., Wang, K., Wang, H., Liu, J., Chen, Y., Feng, J., Sha, C., Peng, X., Lou, Y.: Evaluating large language models in class-level code generation. 2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE), 982–994 (2024)

[4] Ahmad, W., Chakraborty, S., Ray, B., Chang, K.-W.: A transformer-based approach for source code summarization. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp. 4998–5007 (2020)

[5] Yang, Z., Liu, F., Yu, Z., Keung, J.W., Li, J., Liu, S., Hong, Y., Ma, X., Jin, Z., Li, G.: Exploring and unleashing the power of large language models in automated code translation. Proceedings of the ACM on Software Engineering **1**(FSE), 1585–1608 (2024)

[6] Zhang, K., Li, J., Li, G., Shi, X., Jin, Z.: Codeagent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 13643–13658 (2024)

[7] Shankar, S., Zamfirescu-Pereira, J., Hartmann, B., Parameswaran, A., Arawjo, I.: Who validates the validators? aligning llm-assisted evaluation of llm outputs with human preferences. In: Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology, pp. 1–14 (2024)

[8] Sun, T., Xu, J., Li, Y., Yan, Z., Zhang, G., Xie, L., Geng, L., Wang, Z., Chen, Y., Lin, Q., *et al.*: Bitsai-cr: Automated code review via llm in practice. In: Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, pp. 274–285 (2025)

[9] Jiang, N., Li, X., Wang, S., Zhou, Q., Hossain, S.B., Ray, B., Kumar, V., Ma, X., Deoras, A.: Ledex: Training llms to better self-debug and explain code. Advances in Neural Information Processing Systems **37**, 35517–35543 (2024)

[10] Le, T.H., Chen, H., Babar, M.A.: Deep learning for source code modeling and generation: Models, applications, and challenges. ACM Computing Surveys (CSUR) **53**(3), 1–38 (2020)

[11] Yang, C., Liu, Y., Yin, C.: Recent advances in intelligent source code generation: A survey on natural language based studies. Entropy **23**(9), 1174 (2021)

[12] Dehaerne, E., Dey, B., Halder, S., De Gendt, S., Meert, W.: Code generation using machine learning: A systematic review. Ieee Access **10**, 82434–82455 (2022)

[13] Jiang, J., Wang, F., Shen, J., Kim, S., Kim, S.: A survey on large language models for code generation. arXiv preprint arXiv:2406.00515 (2024)

[14] Raihan, N., Newman, C., Zampieri, M.: Code llms: A taxonomy-based survey. In: 2024 IEEE International Conference on Big Data (BigData), pp. 5402–5411 (2024). IEEE

[15] Huawei: Cangjie Programming Languages. https://cangjie-lang.cn/ (2024)

[16] OpenAtom: OpenHarmony. https://www.openharmony.cn/ (2023)

[17] Tarassow, A.: The potential of llms for coding with low-resource and domain-specific programming languages. arXiv preprint arXiv:2307.13018 (2023)

[18] Cassano, F., Gouwar, J., Lucchetti, F., Schlesinger, C., Freeman, A., Anderson, C.J., Feldman, M.Q., Greenberg, M., Jangda, A., Guha, A.: Knowledge transfer from high-resource to low-resource programming languages for code llms. Proceedings of the ACM on Programming Languages **8**(OOPSLA2), 677–708 (2024)

[19] Zhang, J., Zhang, J., Li, Y., Pi, R., Pan, R., Liu, R., Zheng, Z., Zhang, T.: Bridge-coder: Unlocking llms' potential to overcome language gaps in low-resource code. ArXiv **abs/2410.18957** (2024)

[20] Gong, Z., Guo, Y., Zhou, P., Gao, C., Wang, Y., Xu, Z.: Multicoder: Multi-programming-lingual pre-training for low-resource code completion. arXiv preprint arXiv:2212.09666 (2022)

[21] Giagnorio, A., Martin-Lopez, A., Bavota, G.: Enhancing code generation for

low-resource languages: No silver bullet. In: 2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC), pp. 478–488 (2025). IEEE

[22] Joel, S., Wu, J., Fard, F.: A survey on llm-based code generation for low-resource and domain-specific programming languages. ACM New York, NY, ??? (2024)

[23] Baltaji, R., Pujar, S., Mandel, L., Hirzel, M., Buratti, L., Varshney, L.R.: Learning transfers over several programming languages. arXiv preprint arXiv:2310.16937 (2023)

[24] Chen, F., Fard, F.M., Lo, D., Bryksin, T.: On the transferability of pre-trained language models for low-resource programming languages. 2022 IEEE/ACM 30th International Conference on Program Comprehension (ICPC), 401–412 (2022)

[25] Xie, S.M., Raghunathan, A., Liang, P., Ma, T.: An explanation of in-context learning as implicit bayesian inference. arXiv preprint arXiv:2111.02080 (2021)

[26] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., *et al.*: Language models are few-shot learners. Advances in neural information processing systems **33**, 1877–1901 (2020)

[27] Zhuang, F., Qi, Z., Duan, K., Xi, D., Zhu, Y., Zhu, H., Xiong, H., He, Q.: A comprehensive survey on transfer learning. Proceedings of the IEEE **109**, 43–76 (2019)

[28] Wang, Y., Le, H., Gotmare, A., Bui, N., Li, J., Hoi, S.: Codet5+: Open code large language models for code understanding and generation. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pp. 1069–1088 (2023)

[29] Raychev, V., Vechev, M., Yahav, E.: Code completion with statistical language models. In: Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation, pp. 419–428 (2014)

[30] Allamanis, M., Barr, E.T., Devanbu, P., Sutton, C.: A survey of machine learning for big code and naturalness. ACM Computing Surveys (CSUR) **51**(4), 1–37 (2018)

[31] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.D.O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al.: Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 (2021)

[32] Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., Wang, H.: Large language models for software engineering: A systematic literature review. ACM Transactions on Software Engineering and Methodology **33**(8), 1–79 (2024)

[33] Parvez, M.R., Ahmad, W., Chakraborty, S., Ray, B., Chang, K.-W.: Retrieval augmented code generation and summarization, 2719–2734 (2021)

[34] Ogueji, K., Zhu, Y., Lin, J.J.: Small data? no problem! exploring the viability of pretrained multilingual language models for low-resourced languages. Proceedings of the 1st Workshop on Multilingual Representation Learning (2021)

[35] Mora, F., Wong, J., Lepe, H., Bhatia, S., Elmaaroufi, K., Varghese, G., Gonzalez, J.E., Polgreen, E., Seshia, S.A.: Synthetic programming elicitation for text-to-code in very low-resource programming and formal languages. Advances in Neural Information Processing Systems **37**, 105151–105170 (2024)

[36] Xie, Y., Aggarwal, K., Ahmad, A.: Efficient continual pre-training for building domain specific large language models. In: Findings of the Association for Computational Linguistics ACL 2024, pp. 10184–10201 (2024)

[37] Yin, P., Neubig, G.: A syntactic neural model for general-purpose code generation. In: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 440–450 (2017)

[38] Post, M., Vilar, D.: Fast lexically constrained decoding with dynamic beam allocation for neural machine translation. In: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers), pp. 1314–1324 (2018)

[39] DeepSpeed Team: DeepSpeed. https://github.com/deepspeedai/DeepSpeed (2023)

[40] Huawei: Cangjie-SIG. https://gitcode.com/Cangjie-SIG (2024)

[41] Huawei: Cangjie-TPC. https://gitcode.com/Cangjie-TPC (2024)

[42] Huawei: HW-PLLab. https://gitee.com/HW-PLLab (2024)

[43] Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., LIU, S., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., et al.: Graphcodebert: Pre-training code representations with data flow. In: International Conference on Learning Representations

[44] Li, R., Allal, L.B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., Marone, M., Akiki, C., Li, J., Chim, J., Liu, Q., Zheltonozhskii, E., Zhuo, T.Y., Wang, T., Dehaene, O., Davaadorj, M., Lamy-Poirier, J., Monteiro, J., Shliazhko, O., Gontier, N., Meade, N., Zebaze, A.R., Yee, M.-H., Umapathi, L.K., Zhu, J., Lipkin, B., Oblokulov, M., Wang, Z., Murthy, R., Stillerman, J., Patel, S.S., Abulkhanov, D., Zocca, M., Dey, M., Zhang, Z., Fahmy, N., Bhattacharyya, U., Yu, W., Singh, S., Luccioni, S., Villegas, P., Kunakov, M., Zhdanov, F., Romero, M., Lee, T., Timor, N., Ding, J., Schlesinger, C., Schoelkopf, H., Ebert, J., Dao, T., Mishra, M., Gu, A., Robinson, J., Anderson, C.J., Dolan-Gavitt, B., Contractor, D., Reddy, S.,

Fried, D., Bahdanau, D., Jernite, Y., Ferrandis, C.M., Hughes, S.M., Wolf, T., Guha, A., Werra, L., Vries, H.: Starcoder: May the source be with you! arXiv preprint arXiv:2305.06161 (2023)

[45] Team, Q.: Qwen2.5: A Party of Foundation Models (2024). https://qwenlm.github.io/blog/qwen2.5/ Accessed 2025-09-10

[46] Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X., Adi, Y., Liu, J., Remez, T., Rapin, J., Kozhevnikov, A., Evtimov, I., Bitton, J., Bhatt, M.P., Ferrer, C.C., Grattafiori, A., Xiong, W., D'efossez, A., Copet, J., Azhar, F., Touvron, H., Martin, L., Usunier, N., Scialom, T., Synnaeve, G.: Code llama: Open foundation models for code. arXiv preprint arXiv:2308.12950 (2023)

[47] Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., Bi, X., Wu, Y., Li, Y.K., Luo, F., Xiong, Y., Liang, W.: DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence (2024). https://arxiv.org/abs/2401.14196

[48] Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Dal Lago, A., *et al.*: Competition-level code generation with alphacode. Science **378**(6624), 1092–1097 (2022)

[49] Wang, Y., Wang, W., Joty, S., Hoi, S.C.: Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation, 8696–8708 (2021). Association for Computational Linguistics (ACL)

[50] Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al.: Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint arXiv:2501.12948 (2025)

[51] Nag, A., Chakrabarti, S., Mukherjee, A., Ganguly, N.: Efficient continual pre-training of llms for low-resource languages. In: Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: Industry Track), pp. 304–317 (2025)